

Lista de Exercícios — Aula 05

Lista Sequencial Estática

Instruções: As questões a seguir pedem a implementação de novas funções sobre a lista sequencial estática de produtos apresentada na Aula 05. Implemente cada função em C padrão, compile com gcc -Wall -Wextra -std=c11 e não entregue nenhuma função com warning.

As novas funções acessam diretamente os campos qtd e dados da struct lista e devem, portanto, ser implementadas no mesmo arquivo .c em que essa struct está definida.

Formato de entrega da atividade:

Você pode enviar os arquivos .c e .h no Google Classroom ou pode mandar o link para a implementação no seu github.

Biblioteca base (já implementada)

```
#define MAX 100

struct produto {
    int codigo;
    char nome[30];
    float preco;
};

typedef struct lista Lista;

/* já implementadas na Aula 05 */
Lista* cria_lista();
void libera_lista(Lista* li);
int busca_lista_pos(Lista* li, int pos, struct produto *p);
int busca_lista_cod(Lista* li, int cod, struct produto *p);
int insere_lista_final(Lista* li, struct produto p);
int insere_lista_inicio(Lista* li, struct produto p);
int insere_lista_ordenada(Lista* li, struct produto p);
int remove_lista(Lista* li, int cod);
int remove_lista_otimizado(Lista* li, int cod);
int remove_lista_inicio(Lista* li);
int remove_lista_final(Lista* li);
int tamanho_lista(Lista* li);
int lista_cheia(Lista* li);
int lista_vazia(Lista* li);
```

Protótipos a implementar

```
int lista_tem_espaco(Lista* li, int n);
float soma_precos(Lista* li);
int busca_por_nome(Lista* li, char *nome, struct produto *p);
int insere_lista_decrescente(Lista* li, struct produto p);
int remove_mais_caro(Lista* li, struct produto *removido);
int conta_faixa_preco(Lista* li, float min, float max);
```

```
int remove_abaixo_de(Lista* li, float precoMinimo);
int mescla_listas(Lista* destino, Lista* origem);
```

Questão 1

```
int lista_tem_espaco(Lista* li, int n);
```

Implemente a função `lista_tem_espaco`, que devolve 1 se a lista comportar mais `n` inserções sem exceder o tamanho máximo, e 0 caso contrário (incluindo quando `li` é `NULL`).

Observação: Não utilize inserções de teste: a resposta deve vir apenas da comparação entre os campos da lista e o parâmetro `n`.

Questão 2

```
float soma_precos(Lista* li);
```

Implemente a função `soma_precos`, que devolve a soma do campo `preco` de todos os produtos armazenados na lista. Se `li` for `NULL`, a função deve devolver 0.

Questão 3

```
int busca_por_nome(Lista* li, char *nome, struct produto *p);
```

Implemente a função `busca_por_nome`, que busca na lista o primeiro produto cujo campo `nome` seja igual à string apontada por `nome`, e copia esse produto para `*p`. A função devolve 1 em caso de sucesso e 0 caso o nome não seja encontrado ou `li` seja `NULL`.

Observação: Utilize a função `strcmp` da biblioteca `string.h` para comparar as strings.

Questão 4

```
int insere_lista_decrescente(Lista* li, struct produto p);
```

Implemente a função `insere_lista_decrescente`, que insere o produto `p` na lista mantendo-a ordenada de forma decrescente pelo campo `preco`. A função segue a mesma convenção de retorno das demais funções de inserção.

Questão 5

```
int remove_mais_caro(Lista* li, struct produto *removido);
```

Implemente a função `remove_mais_caro`, que localiza o produto de maior preço na lista, remove-o utilizando a técnica de remoção otimizada (Aula 05) e copia o produto removido para `*removido`. A função devolve 1 em caso de sucesso e 0 se a lista estiver vazia ou `li` for `NULL`.

Observação: Em caso de empate entre dois ou mais produtos de maior preço, remova o primeiro deles na ordem em que aparece na lista.

Questão 6

```
int conta_faixa_preco(Lista* li, float min, float max);
```

Implemente a função `conta_faixa_preco`, que devolve quantos produtos da lista têm o campo `preco` entre `min` e `max`, incluindo os dois limites. A função não deve alterar a lista.

Questão 7

```
int remove_abaixo_de(Lista* li, float precoMinimo);
```

Implemente a função `remove_abaixo_de`, que remove da lista, utilizando a técnica de remoção otimizada, todos os produtos cujo preço seja menor que `precoMinimo`. A função devolve o número de produtos removidos.

Observação: Atenção: após a remoção otimizada de uma posição *i*, o elemento que passa a ocupar essa posição ainda não foi conferido pela função. Avaliar essa posição novamente, antes de avançar, é parte do problema.

Questão 8

```
int mescla_listas(Lista* destino, Lista* origem);
```

Implemente a função `mescla_listas`, que insere ao final de destino cada produto de origem cujo código ainda não exista em destino, sem removê-lo de origem. A função respeita o espaço disponível em destino e devolve quantos produtos foram efetivamente inseridos.

Observação: Se um código já existir em destino, o produto correspondente de origem deve ser ignorado, mesmo que os demais campos sejam diferentes.

Observação: Se destino ficar cheio antes de todos os produtos de origem serem avaliados, a função deve parar e devolver a contagem obtida até ali.